

Trabajo práctico Nro 1

Filsinger Gonzalo, Perea Ignacio, Parfait Manuel

*Electrónica Aplicada II 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional*

Resumen—En este informe se abordará el desarrollo del trabajo práctico Nro 1, donde se pondrá en funcionamiento y se programará la BluePill para luego hacerla andar con un led usando sus direcciones GPIO.

Index Terms—BluePill, GPIO, LED, Memoria

I. INTRODUCCIÓN

EN este informe se pondrá en funcionamiento y se programará la placa BluePill en la modalidad BareMetal usando como base códigos ya proporcionados como guías, los cuales se deberán modificar para llevar a cabo dos implementaciones.

II. COMPILACIÓN DE PROGRAMAS EN LA PLACA BLUEPILL

Para programar la placa BluePill se necesitan desarrollar 4 códigos diferentes, cada uno con una función específica en el proceso de compilación y carga del programa en la placa. Estos códigos son:

- **startup: crt.s:** Este código es un archivo en ensamblador y es el encargado de configurar el entorno de ejecución para la placa. Incluyendo la inicialización de la memoria, la configuración de los registros necesarios, define los valores correspondientes a los vectores de interrupción, inicializa el stack y define la direcciones de memoria para vincular el archivo main.c.
- **linker: linker.ld:** es un script de enlace que indica al enlazador cómo los diferentes archivos objetos y librerías deben ser combinados. Define las posiciones de la memoria de variables y de programa, y la posición de la primera instrucción. Es esencial para asegurar que el programa se cargue correctamente en la memoria del microcontrolador.
- **Programa: main.c:** Este es el código principal del programa, donde se implementa la lógica específica para llevar a cabo una tarea determinada.
- **Makefile:** Con la herramienta Make, se automatiza el proceso de compilación de los tres códigos anteriores, el Makefile define las reglas para compilar cada uno de los archivos y que termine generando un archivo ejecutable que se pueda cargar en la placa BluePill.

La compilación de estos códigos se realiza utilizando el compilador arm-none-eabi-gcc, que es una herramienta de

compilación específica para microcontroladores ARM. La siguiente imagen muestra la compilación de nuestro proyecto utilizando make.

```
~/Tecnica2/tpl
> make
arm-none-eabi-as -o build/crt.o src/crt.s
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -O0 -g -c -o build/main.o src/main.c
arm-none-eabi-ld -T src/linker.ld -o build/app.elf build/crt.o build/main.o
arm-none-eabi-objcopy -O binary build/app.elf build/app.bin

~/Tecnica2/tpl
> make openocd-flash
arm-none-eabi-as -o build/crt.o src/crt.s
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -O0 -g -c -o build/main.o src/main.c
arm-none-eabi-ld -T src/linker.ld -o build/app.elf build/crt.o build/main.o
arm-none-eabi-objcopy -O binary build/app.elf build/app.bin
openocd -f interface/stlink.cfg -c "set CPUID 0x2ba01477" -f target/stm32fx.cfg \
-c "init" -c "reset halt" \
-c "flash write_image erase build/app.bin 0x08000000" \
-c "reset run" -c "exit"
Open On-Chip Debugger 0.12.0-01004-g9ea7f3d64-dirty (2025-11-12-08:18)
Licensed under GNU GPL v2
For bug reports, read
http://openocd.org/doc/doxygen/bugs.html
0x2ba01477
Info : auto-selecting first available session transport "hla_swd". To override use 'transport select <transport>'.
Info : The selected transport took over low-level target control. The results might differ compared to plain JTAG/SWD
Info : clock speed 1000 kHz
Info : SWJCLK_V22957 (API v2) VID:PID 0483:3748
Info : Target voltage: 3.251665
Info : [stm32fx.cpu] Cortex-M3 r2p8 processor detected
Info : [stm32fx.cpu] target has 6 breakpoints, 4 watchpoints
Info : starting gdb server for stm32fx.cpu on 3333
Info : Listening on port 3333 for gdb connections
[stm32fx.cpu] halted due to debug-request, current mode: Thread
xPSR: 0x01000000 pc: 0x0800018e msp: 0x20005000
Info : device id = 0x10010414
Info : Flash size = 256 KiB
Warn : Adding extra erase range, 0x08000194 .. 0x080007ff
auto erase enabled
wrote 404 bytes from file build/app.bin in 0.004677s (4.659 KiB/s)
```

Compilación del proyecto utilizando make

Para el funcionamiento de la placa clon, se tuvieron que cambiar los comandos de programación y depuración, además de modificar el Makefile para añadir dichos comandos y así que funcione de manera correcta.

III. PARPADEO DE LEDS: GPIO E INTEGRADO EN PLACA

Para este programa se usará el código de guía *main.c* el cual ya hace parpadear el led de la placa, las modificaciones que se le harán a este código serán para hacer parpadear en conjunto con este otro led externo conectado a la placa en una GPIO.

```
#include <stdint.h>

// register address
#define RCC_BASE 0x40021000
#define GPIOC_BASE 0x40011000
#define GPIOA_BASE 0x40010800

#define RCC_APB2ENR *(volatile uint32_t *) (RCC_BASE + 0x18)
#define GPIOC_CRH *(volatile uint32_t *) (GPIOC_BASE + 0x04)
#define GPIOA_CRL *(volatile uint32_t *) (GPIOA_BASE + 0x00)
#define GPIOC_ODR *(volatile uint32_t *) (GPIOC_BASE + 0x0C)
#define GPIOA_ODR *(volatile uint32_t *) (GPIOA_BASE + 0x0C)

// bit fields
#define RCC_IOPCEN (1 << 4)
#define RCC_IOPAEN (1 << 2)
#define GPIOC13 (1UL << 13)
#define GPIOA7 (1UL << 7)
```

Definiciones de los registros en main.c

Aquí se configuran los registros necesarios para configurar los pines de la placa y su modo de funcionamiento. Las modificaciones que se realizaron para configurar el pin GPIOA7 fueron las siguientes:

- Se definió el registro **GPIOA_BASE** con la dirección de memoria del puerto A, la cual es 0x40010800.
- Se definió el registro **GPIOA_CRL**, el registro CRL (Control Register Low) se utiliza para configurar los pines del puerto A desde el pin 0 al pin 7, el registro CRL siempre tiene un offset de 0x00 y sirve para determinar el modo de funcionamiento de cada pin.
- Se definió el registro **GPIOA_ODR** para escribir el estado del pin, el registro ODR (Output Data Register) tiene un offset de 0x0C ya que se utiliza para escribir el estado del pin cuando este se encuentra configurado como salida.
- Se definió la máscara **RCC_IOPAEN** la cual se utiliza para habilitar el reloj del puerto A, esta máscara tiene un corrimiento de 2 bits ya que el bit 2 del registro **RCC_APB2ENR** se utiliza para habilitar el reloj del puerto A.
- Se definió la máscara **GPIOA7** la cual se utiliza para configurar el pin A7 con un corrimiento de 7 bits. Se utiliza Unsigned long para evitar overflows al hacer el corrimiento de bits.

Luego, en la función **main** se realizaron las siguientes modificaciones:

```
void main(void)
{
    RCC_APB2ENR |= RCC_IOPCEN;
    RCC_APB2ENR |= RCC_IOPAEN;
    GPIOC_CRH &= 0xFF0FFFFF;
    GPIOC_CRH |= 0x00200000;
    GPIOA_CRL &= 0x0FFFFFFF;
    GPIOA_CRL |= 0x20000000;

    while (1)
    {
        GPIOC_ODR &= ~GPIOC13;
        delay_ms_bloqueante(500);
        GPIOC_ODR |= GPIOC13;
        delay_ms_bloqueante(500);

        GPIOA_ODR |= GPIOA7;
        delay_ms_bloqueante(500);
        GPIOA_ODR &= ~GPIOA7;
        delay_ms_bloqueante(500);
    }
}
```

Modificaciones en la función main

- En **RCC_APB2ENR** se habilitó el reloj del puerto A utilizando la máscara **RCC_IOPAEN**.
- En **GPIOA_CRL &=** se ponen en 0 los cuatro bits correspondientes al pin A7 como medida de limpieza.
- En **GPIOA_CRL |=** se configura el pin A7 como salida en modo Push-Pull a una velocidad de 2 MHz (b0010).
- En el ciclo "while" se agregaron dos líneas para escribir el estado del pin A7 utilizando el registro **GPIOA_ODR**. Para encender el led se utiliza la operación OR con la máscara **GPIOA7** y para apagarlo se utiliza la operación AND con la negación de la máscara **GPIOA7**.

La utilización de la función **delay_ms_bloqueante** es para generar un retardo entre el encendido y apagado de los leds.

IV. RETARDO POR INTERRUPCIONES

Ahora se busca generar un retardo mediante interrupciones SysTick, para esto se modificará el código de la parte anterior para generar un retardo utilizando interrupciones. Las modificaciones realizadas para generar el retardo por interrupciones fueron las siguientes:

```
// --- Registros del SysTick (Cortex-M Core) ---
#define SysTick_BASE 0xE000E010
#define SysTick_CTRL (*(volatile uint32_t *) (SysTick_BASE + 0x00))
#define SysTick_LOAD (*(volatile uint32_t *) (SysTick_BASE + 0x04))
#define SysTick_VAL (*(volatile uint32_t *) (SysTick_BASE + 0x08))

// Bits de control de SysTick
#define SysTick_CTRL_ENABLE (1 << 0) // Habilita el contador
#define SysTick_CTRL_TICKINT (1 << 1) // Habilita la interrupción de SysTick
#define SysTick_CTRL_CLKSOURCE (1 << 2) // Fuente de reloj
#define SysTick_CTRL_COUNTFLAG (1 << 16) // Bandera de desborde

volatile uint32_t tick;
```

Definiciones de los registros para interrupciones

- Se definió el registro **SYSTICK_BASE** con la dirección de memoria del periférico SysTick, la cual es 0xE000E010.
- Se definió el registro **SYSTICK_CTRL** el cual se utiliza para controlar el funcionamiento del SysTick.
- Se definió el registro **SYSTICK_LOAD** el cual guarda el valor inicial desde donde empezará la cuenta regresiva.
- Se definió el registro **SYSTICK_VAL** el cual guarda el valor actual del contador, este registro se puede leer para conocer el valor actual del contador.
- Se definió la máscara **SYSTICK_ENABLE** la cual se utiliza para habilitar el contador del SysTick, al ponerse en 1 el contador comenzará a descontar desde el valor cargado en el registro SYSTICK_LOAD.
- Se definió la máscara **SYSTICK_TICKINT** la cual se utiliza para habilitar las interrupciones del SysTick, al ponerse en 1 se habilitan las interrupciones y al llegar a 0 el contador generará una interrupción.
- Se definió la máscara **SYSTICK_CLKSOURCE** la cual se utiliza para seleccionar la fuente de reloj del SysTick, al ponerse en 1 se selecciona el reloj del procesador (HCLK) y al ponerse en 0 se selecciona el reloj del procesador dividido 8 (HCLK/8).
- Se definió la máscara **SYSTICK_COUNTFLAG** la cual se utiliza para verificar si el contador ha llegado a 0, al leer este bit se pone en 0 automáticamente.
- Se definió la variable global **tick** la cual se utiliza para contar la cantidad de interrupciones generadas por el SysTick, esta variable se incrementa en cada interrupción y se utiliza para generar el retardo deseado.

Luego se agregaron las siguientes funciones para configurar el SysTick y generar el retardo por interrupciones:

```
// función que atiende la interrupción
void SysTick_Handler(void)
{
    tick++;
}

// inicialización del systick
void systick_init_ms(void)
{
    tick = 0;
    SysTick_CTRL &= ~SysTick_CTRL_CLKSOURCE; // Reloj de SysTick
    SysTick_LOAD = 999; // Poner valor de RECARGA para 1000 ticks
    SysTick_VAL = 0; // reset del contador
    SysTick_CTRL |= SysTick_CTRL_TICKINT | SysTick_CTRL_ENABLE; //
}

// función delay
void delay_ms_bloqueante(uint32_t ms)
{
    uint32_t tiempo_inicio = tick + ms;
    // Espera activa hasta que el contador global haya avanzado
    while (tick != tiempo_inicio);
}
```

Funciones para configurar el SysTick y generar el retardo por interrupciones

- La función **SysTick_Handler** es la rutina de servicio de interrupción del SysTick, esta función se ejecuta cada vez que se genera una interrupción, en esta función se incrementa la variable global **tick** para contar la cantidad de interrupciones generadas.
- La función **systick_init_ms** es la función que se utiliza para configurar el SysTick para generar interrupciones cada cierto tiempo. Esta pone en 0 la variable tick cada vez que se inicia, hace que se divida el reloj del procesador en 8, se configura el disparo en 1ms, inicializa la variable VAL en 0 y habilita el contador y las interrupciones del SysTick.
- La función **delay_ms_bloqueante** es la función que se utiliza para hacer parpadear los leds utilizando el retardo por interrupciones.

Con esto en cuenta lo que se modificó en la función **main** para generar el retardo por interrupciones fue lo siguiente:

```

systick_init_ms();
while (1)
{

    GPIOC_ODR &= ~GPIOC13;
    delay_ms_bloqueante(500);
    GPIOC_ODR |= GPIOC13;
    delay_ms_bloqueante(500);

    GPIOA_ODR |= GPIOA7;
    delay_ms_bloqueante(500);
    GPIOA_ODR &= ~GPIOA7;
    delay_ms_bloqueante(500);
}

```

Función main con retardo por interrupciones

V. CONCLUSIÓN

Se logró poner en funcionamiento la BluePill, y se logró hacer andar un led usando sus direcciones GPIO. Para esto se tuvo que usar los comandos para la placa clon, además de modificar el Makefile para añadir dicho comando y así que funcione de manera correcta. Lo más difícil de este trabajo fue comenzar a pensar el código en base a los registros, ya que es un nivel de abstracción al que no estábamos acostumbrados, pero una vez que logramos comprender el funcionamiento del código logramos comprender mejor como funciona un microcontrolador a nivel de hardware, nos resultó particularmente interesante como un programa de alto nivel pasa a ser encender o apagar determinados bits de ciertos registros.

VI. CODIGOS

VI-A. main.c

```

1 #include <stdint.h>
2 // register address
3 #define RCC_BASE 0x40021000
4 #define GPIOC_BASE 0x40011000
5 #define GPIOA_BASE 0x40010800
6 #define RCC_APB2ENR *(volatile uint32_t *) (RCC_BASE
7 + 0x18)
8 #define GPIOC_CRH *(volatile uint32_t *) (GPIOC_BASE
9 + 0x04)
10 #define GPIOA_CRL *(volatile uint32_t *) (GPIOA_BASE
11 + 0x00)
12 #define GPIOC_ODR *(volatile uint32_t *) (GPIOC_BASE
13 + 0x0C)
14 #define GPIOA_ODR *(volatile uint32_t *) (GPIOA_BASE
15 + 0x0C)
16 // bit fields
17 #define RCC_IOPCEN (1 << 4)
18 #define RCC_IOPAEN (1 << 2)
19 #define GPIOC13 (1UL << 13)

```

```

15 #define GPIOA7 (1UL << 7)
16 // --- Registros del SysTick (Cortex-M Core) ---
17 #define SysTick_BASE 0xE000E010
18 #define SysTick_CTRL *(volatile uint32_t *) (
19 SysTick_BASE + 0x00)
20 #define SysTick_LOAD *(volatile uint32_t *) (
21 SysTick_BASE + 0x04)
22 #define SysTick_VAL *(volatile uint32_t *) (
23 SysTick_BASE + 0x08)
24 // Bits de control de SysTick
25 #define SysTick_CTRL_ENABLE (1 << 0) // Habilita
26 el contador
27 #define SysTick_CTRL_TICKINT (1 << 1) // Habilita
28 la interrupción de SysTick
29 #define SysTick_CTRL_CLKSOURCE (1 << 2) // Fuente de
30 reloj
31 #define SysTick_CTRL_COUNTFLAG (1 << 16) // Bandera
32 de desborde
33 volatile uint32_t tick;
34 // función que atiende la interrupción
35 void SysTick_Handler(void)
36 {
37     tick++;
38 }
39 // inicialización del systick
40 void systick_init_ms(void)
41 {
42     tick = 0;
43     SysTick_CTRL &= ~SysTick_CTRL_CLKSOURCE; //
44     Reloj de SysTick a HCLK/8 (8MHz / 8 = 1MHz -> 1
45     tick = 1us)
46     SysTick_LOAD = 999; // Poner valor de RECARGA
47     para 1000 ticks (1ms) (1000 - 1) = 999
48     SysTick_VAL = 0; // reset del contador
49     SysTick_CTRL |= SysTick_CTRL_TICKINT |
50     SysTick_CTRL_ENABLE; // Habilitar la interrupció
51     n de SysTick (TICKINT) Y el contador (ENABLE)
52 }
53 // función delay
54 void delay_ms_bloqueante(uint32_t ms)
55 {
56     uint32_t tiempo_inicio = tick + ms;
57     // Espera activa hasta que el contador global
58     haya avanzado 'ms' milisegundos
59     while (tick != tiempo_inicio);
60 }
61 void main(void)
62 {
63     RCC_APB2ENR |= RCC_IOPCEN;
64     RCC_APB2ENR |= RCC_IOPAEN;
65     GPIOC_CRH &= 0xFFFF0000;
66     GPIOC_CRH |= 0x00200000;
67     GPIOA_CRL &= 0xFFFF0000;
68     GPIOA_CRL |= 0x20000000;
69     systick_init_ms();
70     while (1)
71     {
72         GPIOC_ODR &= ~GPIOC13;
73         delay_ms_bloqueante(500);
74         GPIOC_ODR |= GPIOC13;
75         delay_ms_bloqueante(500);

76         GPIOA_ODR |= GPIOA7;
77         delay_ms_bloqueante(500);
78         GPIOA_ODR &= ~GPIOA7;
79         delay_ms_bloqueante(500);
80     }
81 }

```

Listing 1. Configuración inicial en main.c

VI-B. crt.s

```

1 .syntax unified
2 .cpu cortex-m3
3 .thumb
4 .extern _esstack
5
6 .section .text.manejadores // definimos la seccion
   text.manejadores
7
8 // función básica para manejar los punteros
9 .thumb_func
10 .weak Default_Handler
11 Default_Handler:
12     b . // Bucle infinito */
13
14 // definir los vectores ejemplo
15 .weak NMI_Handler
16 .thumb_set NMI_Handler, Default_Handler
17 .weak SysTick_Handler
18 .thumb_set SysTick_Handler, Default_Handler
19 .weak HardFault_Handler
20 .thumb_set HardFault_Handler, Default_Handler
21
22 // vectores
23 .section .isr_vector, "a", %progbits
24 .word _esstack
25 .word _reset
26 .word NMI_Handler // 2: NMI (Non-Maskable
   Interrupt) */
27 .word HardFault_Handler // 3: HardFault (falla
   grave) */
28 .word 0 // 4: MemManage (falla de
   memoria) */
29 .word 0 // 5: BusFault (falla de
   bus) */
30 .word 0 // 6: UsageFault (falla
   de uso) */
31 .word 0 // 7: Reservado */
32 .word 0 // 8: Reservado */
33 .word 0 // 9: Reservado */
34 .word 0 // 10: Reservado */
35 .word 0 // 11: SVCALL (Llamada al
   Supervisor) */
36 .word 0 // 12: Debug Monitor */
37 .word 0 // 13: Reservado */
38 .word 0 // 14: PendSV (Interrupció
   n pendiente del sistema) */
39 .word SysTick_Handler // 15: El handler del
   SysTick */
40
41 // declaramos la función _reset dentro de la seccion
   text.reset
42 .section .text.reset
43 .thumb_func
44 .global _reset
45 _reset:
46     bl main
47     b .

```

VI-C. linker.ld

```

1 MEMORY
2 MEMORY
3 {
4     FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K
5     RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 20K
6 }
7
8 /* 2. Definición de Símbolos Globales */
9 ENTRY(_reset) /* El punto de entrada, definido en tu
   crt.s */
10
11 /* Símbolo para el final de la pila, usado en tu .
   isr_vector */
12 _esstack = ORIGIN(RAM) + LENGTH(RAM); /* 0x20005000
   (el final de los 20K de RAM) */

```

```

13 /* 3. Definición de las Secciones de Salida */
14 SECTIONS
15 {
16     /* SECCIONES EN FLASH */
17     /* La tabla de vectores debe ir primero en la
   Flash */
18     .isr_vector :
19     {
20         KEEP(*(.isr_vector)) /* Mantiene la sección .
   isr_vector de tu crt.s */
21     } > FLASH
22
23     /* Código del programa y datos de solo lectura */
24     .text :
25     {
26         *(.text) /* Todo el código C y
   ensamblador */
27         *(.text*) /* Variantes de .text */
28         *(.rodata) /* Datos de solo lectura (
   constantes) */
29         *(.rodata*)
30     } > FLASH
31
32     /* SECCIONES EN RAM */
33     /* .data: Variables inicializadas con un valor
   distinto de cero (ej: int x = 10;) */
34     /* El linker necesita símbolos para que el startup
   mueva los datos */
35     _sidata = LOADADDR(.data); /* Dirección de los
   valores en FLASH */
36
37     .data :
38     {
39         _sdata = .; /* Inicio de .data en RAM
   */
40         *(.data)
41         *(.data*)
42         _edata = .; /* Fin de .data en RAM */
43     } > RAM AT> FLASH /* Ubicada en RAM, pero
   cargada desde FLASH */
44     /* .bss: Variables inicializadas a cero o sin
   inicializar */
45     /* (ej: static int y; o int z = 0;) */
46     .bss :
47     {
48         _sbss = .; /* Símbolo de inicio de .
   bss */
49         *(.bss)
50         *(.bss*)
51         *(COMMON)
52         _ebss = .; /* Símbolo de fin de .bss
   */
53     } > RAM /* Solo ocupa espacio en RAM
   */
54 }

```

VI-D. makefile

```

1 SRC_DIR = src
2 BUILD_DIR = build
3 CC = arm-none-eabi-gcc
4 AS = arm-none-eabi-as
5 LD = arm-none-eabi-ld
6 BIN = arm-none-eabi-objcopy
7 CFLAGS = -mthumb -mcpu=cortex-m3 -o0 -g
8
9 all: app.bin
10
11 $(BUILD_DIR):
12     mkdir $(BUILD_DIR)
13
14 crt.o: $(BUILD_DIR) $(SRC_DIR)/crt.s
15     $(AS) -o $(BUILD_DIR)/crt.o $(SRC_DIR)/crt.s
16
17 main.o: $(BUILD_DIR) $(SRC_DIR)/main.c

```

```
18     $(CC) $(CFLAGS) -c -o $(BUILD_DIR)/main.o $(
19     SRC_DIR)/main.c
20 app.elf: $(BUILD_DIR) $(SRC_DIR)/linker.ld crt.o
21     main.o
22     $(LD) -T $(SRC_DIR)/linker.ld -o $(BUILD_DIR)/
23     app.elf $(BUILD_DIR)/crt.o $(BUILD_DIR)/main.o
24
25 app.bin: app.elf
26     $(BIN) -O binary $(BUILD_DIR)/app.elf $(
27     BUILD_DIR)/app.bin
28
29 clean:
30     rm -rf $(BUILD_DIR)
31
32 openocd-flash: app.bin
33     openocd -f interface/stlink.cfg -c "set CPUTAPID
34     0x2ba01477" -f target/stm32f1x.cfg \
35     -c "init" -c "reset halt" \
36     -c "flash write_image erase $(BUILD_DIR)/app.bin
37     0x08000000" \
38     -c "reset run" -c "exit"
39
40 openocd-erase:
41     openocd -f interface/stlink.cfg -c "set CPUTAPID
42     0x2ba01477" -f target/stm32f1x.cfg \
43     -c "init" -c "reset halt" \
44     -c "flash erase_sector 0 0 last" -c "reset run"
45     -c "exit"
```